

WatchSmart AI Launch Execution Document - Detailed Build Brief

One detailed document for Replit AI / engineering to make the app beta-ready

MASTER INSTRUCTION TO AI / ENGINEERING AGENT

You are making WatchSmart beta-launch ready. Do not redesign the app. Do not add non-essential new features. Work phase by phase, starting with P0 launch blockers. After each phase, report: files changed, database fields changed, environment secrets needed, automated tests run, manual tests run, pass/fail evidence, and remaining blockers.

Core product promise:

WatchSmart helps users find good films and shows they can actually watch on the services they already pay for. Discover is the personalized, subscription-aware recommendation feed. Search is a universal catalogue lookup and must search the full database with clear availability labels, not hide results by default.

Critical rule:

Do not merge Discover and Search logic.

- Discover = personalized, subscription-aware, quality-gated recommendation feed.
- Search = full catalogue lookup with optional filters and availability labels.

Do not claim launch-ready unless these pass: accounts/session persistence, swipe speed, card state, full catalogue search, provider accuracy, strict subscription trust, user-data persistence, and personalization after real swipes.

Priority Map

Priority	Area	Beta blocker	Why it matters
P0	Accounts/session/persistence	Yes	All user learning and saved data depend on stable identity.
P0	Swipe speed/feed queue	Yes	Blank screens kill the swipe UX.
P0	Discover state/card persistence	Yes	Cards disappearing breaks trust and usability.
P0	Strict subscription trust	Yes	This is the main product promise.
P0	Provider accuracy by region	Yes	Wrong availability destroys trust.
P0	Full catalogue search	Yes	Users expect title, actor, genre, provider, and TV search.
P1	Personalization learning	Yes for beta	Swipes must visibly change recommendations.
P1	Catalogue quality/metadata	Yes for beta	Bad data makes the engine feel random.
P2	Apple/Google sign-in	No	Defer until email/password accounts are stable.
P2	Friends/social	No	Later feature after core is reliable.

P0 - Accounts, Auth, Sessions, Emails, and User Data Persistence

Why this matters: All recommendation learning depends on stable identity. If accounts are unstable, swipes, watchlists, filters, and taste profiles become useless.

Likely files / areas to inspect:

- backend/routers/auth.py
- frontend/src/context/AuthContext.jsx
- frontend/src/pages/Login.jsx

- frontend/src/pages/Register.jsx
- user model/schema
- email service utility
- admin routes/dashboard

Implementation tasks:

- Implement reliable email/password signup and login first. Do not implement Apple/Google sign-in in this phase.
- Passwords must be securely hashed. Duplicate emails must be blocked.
- Users must stay logged in after refresh, browser reopen, and long sessions. Logout must clear local/session state cleanly.
- Add global 401 handling: stop request loops, clear invalid auth, redirect safely or show friendly session-expired message, never show raw Axios/runtime errors.
- Persist per user: email, name, DOB/age range if collected, gender if collected, terms/privacy acceptance timestamp, selected subscriptions, onboarding choices, filters, saved/liked, skipped/disliked, watched/seen, shown history, action history, search history if used, trailer opens, detail opens, taste weights, genre weights, language weights, decade weights, theme weights, tone weights, pacing weights, exploration weight.
- Add welcome email after signup from hello@watchsmart.uk or support@watchsmart.uk.
- Add forgot-password and reset-password email flow. Reset tokens must expire and cannot be reused.
- Store email provider API keys only in environment secrets. Never expose secrets in frontend code or logs.
- Add protected admin-only visibility for user count, lookup by email, user taste debug, catalog health, provider health, recommendation debug. Normal users must be blocked.

Acceptance tests:

- Create account -> complete onboarding -> perform 50 actions -> refresh -> close/reopen -> logout/login -> all user data remains.
- Long session does not randomly log user out.
- Expired token produces friendly recovery, not raw error overlay.
- Password reset email arrives, token expires, password changes securely.
- Admin can see debug; non-admin gets blocked.

P0 - Swipe Speed, Queue Depth, Preloading, and Blank Screen Fix

Why this matters: Swiping must feel instant. Blank screens or waiting for backend kills the core UX.

Likely files / areas to inspect:

- frontend/src/pages/Discover.jsx
- backend/routers/discovery.py
- backend/engine.py
- API client/cache utility

Implementation tasks:

- Increase buffers: initial Discover queue 150-200 where possible; refill threshold 50-60; refill batch 100-150; image preload ahead 12-15 cards.
- Do not refresh the full user object after every swipe. Debounce or batch refresh every 10 swipes or every 30-60 seconds unless immediate data is required.
- Cards must be served from local queue. Refill happens in the background before stack is low.
- Preload poster/backdrop images for upcoming cards. Show skeleton only on first load or true failure, not between normal swipes.
- Add performance debug fields to feed response/UI: current stack size, queue size, refill in progress, image preload count, API response time, render time, refill returned count, refill rejected count.
- Protect against refill races that clear the stack or append duplicates incorrectly.

Acceptance tests:

- 100 rapid swipes with no blank screen.
- 300 swipe session with refills and no waiting for backend between normal swipes.

- Slow network simulation still shows next cards from queue.
- Backend delay does not freeze the feed while queue has cards.

P0 - Discover State and Card Persistence

Why this matters: Users must be able to tap a card, inspect it, and return without losing place or state.

Likely files / areas to inspect:

- frontend/src/pages/Discover.jsx
- MovieDetail component/route
- navigation/cache utilities

Implementation tasks:

- Opening MovieDetail must not count as skip/save/watched and must not remove the card.
- Detail open may track engagement only.
- Returning from MovieDetail must restore current stack, queue, selected tab, filters, subscription mode, include rent/buy toggle, include other services toggle, empty-state reason/banner, and card position.
- Do not clear sessionStorage/cache on detail open.
- Only remove a card after explicit action: save/like, skip/dislike, watched/seen, or swipe.
- Prevent refill from firing incorrectly just because user returns from MovieDetail.

Acceptance tests:

- Open/back on 20 cards without losing card state.
- Turn on include rent/buy, open card, go back, toggle and feed remain.
- Refresh during detail then recover safely.

P0 - Search as Full Catalogue Lookup

Why this matters: Search must behave like IMDb/JustWatch style lookup, not a filtered Discover feed.

Likely files / areas to inspect:

- backend/routers/content.py or search router
- frontend/src/pages/Search.jsx
- backend/cast_backfill.py
- catalogue schema/indexes

Implementation tasks:

- Search must query the full backend database, not Discover/feed/local state.
- Search must include movies and TV shows.
- Search must support title, original title, aliases, actor/cast, director, studio/company, genre, year, provider, keyword/theme, franchise/collection, and fuzzy typo matching.
- By default, Search includes unavailable, outside-subscription, and rent/buy results, but labels availability clearly.
- Do not automatically hide results outside selected subscriptions in Search.
- Add optional filter chips: My subscriptions, Hide unavailable, Movies, TV, Genre, Year, Provider.
- Add pagination/infinite scroll and total result count.
- Populate/index cast_names, director_names, studio_names, collections, aliases, and keywords in movies_cache.
- Run/repair cast/director/theme backfill across the catalogue.

Acceptance tests:

- Search tests: Simpsons, The Simpsons Movie, Tom Holland, Leonardo DiCaprio, Christopher Nolan, A24, Studio Ghibli, Netflix horror, HBO Max, Band of Brothers, The White Lotus, Harry Potter, vampire, zombie, 90s comedy.
- Actor searches return actor-related titles.
- Provider searches return provider-related titles.

- Results display labels: Included with your subscriptions, Available elsewhere, Rent/buy, Unavailable.

P0 - Provider Accuracy, Max/HBO, Regions, and Availability Trust

Why this matters: Wrong streaming availability destroys trust because the app promises content users can watch.

Likely files / areas to inspect:

- backend/routers/content.py
- provider sync files
- backend/catalog_import.py
- MovieDetail component
- provider alias map

Implementation tasks:

- Store provider data by region: providers.GB, providers.US, providers.CA, providers.AU, etc.
- Use user country/region for Discover, MovieDetail, and availability labels. Do not assume US or GB globally.
- Do not set providers_fetched=true after failed, empty, rate-limited, stale, partial, or invalid provider fetches.
- Add on-demand provider refresh for missing/stale records.
- Add provider refresh timestamps and provider confidence scores.
- Normalize aliases: Max, HBO Max, HBO, HBO Go, Prime Video, Amazon Video, Apple TV, Apple TV Plus, Disney Plus, Netflix.
- Re-enrich TV shows as well as movies. Check show-level and season-level availability.
- Add user report issue flow for wrong provider, missing title, bad metadata, duplicate, or unavailable title.
- Add admin provider health report: provider counts by region and content type, stale count, missing count, confidence distribution.

Acceptance tests:

- Known Max/HBO tests by region: Band of Brothers, The Sopranos, Game of Thrones, The Last of Us, House of the Dragon, Chernobyl, The White Lotus, A Knight of the Seven Kingdoms.
- Known Netflix, Prime, Disney, Apple, rent/buy examples pass.
- Region tests: UK, US, Canada, Australia.

P0 - Catalogue Ingestion and Metadata Completeness

Why this matters: The engine cannot recommend/search what is not in movies_cache. Incomplete metadata makes learning feel random.

Likely files / areas to inspect:

- backend/catalog_import.py
- backend/cast_backfill.py
- provider sync/import files
- movies_cache schema/indexes

Implementation tasks:

- Expand catalogue ingestion beyond popular/top-rated/trending pages.
- Add provider-first ingestion by region for Netflix, Prime Video, Disney+, Apple TV+, Max/HBO, Hulu, Paramount+, Peacock, YouTube/Amazon rental sources.
- Store full metadata: cast, directors, studios, collections, franchises, keywords, themes, production countries, original language, spoken languages, runtime, release date, status, provider data, trailers, vote count, rating, popularity, posters, backdrops.
- Do not hard block old films at import. Old films must be searchable.
- Move old-film handling to Discover ranking/eligibility. Default Discover should downrank pre-1995/pre-2000 for new users unless Classics is enabled or behavior indicates interest.

- Add Discover quality gate: poor metadata, missing artwork, missing overview, missing runtime, unreleased/planned/in production, very low vote count/rating, and no provider data should not appear by default.
- Keep weak/old/niche titles searchable even if not recommended by default.
- Add catalogue health report: total titles, titles per genre, titles per provider/region, cast coverage, director coverage, keyword/theme coverage, provider coverage, poster coverage, trailer coverage, runtime coverage, stale provider records, low-quality recommendation-eligible count.

Acceptance tests:

- Old classics appear in Search but not random Discover for modern users.
- Actor/director search improves after backfill.
- Provider-first ingestion increases Max/HBO and other service pools.
- Catalogue health report improves coverage percentages.

P1 - Recommendation Engine, Action Semantics, and Swipe Learning

Why this matters: The engine exists, but signal interpretation must be tuned so behavior visibly changes recommendations.

Likely files / areas to inspect:

- backend/core.py
- backend/engine.py
- backend/routers/user.py
- frontend action handlers

Implementation tasks:

- Confirm frontend action names match backend. If UI sends like/dislike, backend must handle like/dislike, not only save/skip/watched.
- Action meaning: Like/Save = strongest positive. Not for me/Skip = strong negative. Watched/Seen = remove from feed and weak/neutral signal. Watched + liked = strongest positive. Watched + not for me = strongest negative.
- Negative signals should be stronger than one-off positives for languages, countries, decades, tones, documentary subtypes, anime, family/kids, holiday content, and repeated skips.
- Recent behavior should outweigh onboarding after enough actions.
- 20-30 swipes should create noticeable change. 50 swipes should create strong change. 100 swipes should feel clearly personalized.
- Reduce fixed randomness. RANDOM_INJECTION_PCT around 0.08 is too high for confident/niche users. Make randomness adaptive: higher for new users, lower for high-confidence users.
- Adjacent/wildcard logic must obey learned dislikes and tone compatibility.
- Add suppression/debug endpoint: top liked themes, top disliked themes, blocked languages, blocked decades, tone suppression, why card was shown, why candidate was rejected.
- Ensure feed queue rebuilds or adapts after enough new user actions. Do not keep serving old cached queue that ignores new learning.

Acceptance tests:

- Modern English horror user: after 50 swipes, Japanese/international/old/light content drops sharply.
- Superhero-only user: superhero/comic-book/antihero theme rises, not just Action/Sci-Fi.
- User skips old films repeatedly: older decades reduce or block.
- User marks Seen/Watched only: remove from feed without treating as strong positive.

P1 - Dynamic Theme, Subgenre, and Taste Fingerprint System

Why this matters: Users like specific taste patterns, not only broad genres.

Likely files / areas to inspect:

- backend/engine.py
- catalog enrichment/backfill files

- user taste profile schema

Implementation tasks:

- Do not hardcode only superhero or a tiny list. Build dynamic theme extraction from TMDB keywords, overview text, genres, collections, franchises, production companies, actors/directors, language/country, and later AI enrichment.
- Store normalized themes/subgenres on each title.
- Examples include superhero, comic-book, psychological horror, slasher, supernatural, true crime, serial killer, spy, heist, revenge, courtroom, prison, sports documentary, music documentary, nature documentary, time travel, dystopian, gangster, mafia, survival, disaster, martial arts.
- Theme weights must count positive and negative signals. Do not only score positive themes. Disliked themes must suppress candidates.
- If exact niche pool is thin, widen by taste fingerprint rather than random genre expansion.
- Fallback ladder: exact niche, same genre and tone, same themes, same pacing/runtime, same audience/mood, adjacent genre, broader exploration.
- Every user should have a personal pool built from shared catalogue plus individual taste profile.

Acceptance tests:

- Superhero-only persona: superhero exposure increases strongly after repeated likes.
- Psychological horror persona: expands to dark psychological thrillers, not random crime/comedy.
- True crime documentary persona: narrows into true crime/sports/history depending on reactions.

P1 - Subscription Modes and Product Trust

Why this matters: The default promise is subscriptions only, but users need explicit broader modes if they want them.

Likely files / areas to inspect:

- backend/engine.py
- backend/routers/discovery.py
- frontend/src/pages/Discover.jsx
- Filters/preferences components

Implementation tasks:

- Default viewing mode: My subscriptions only.
- Optional modes: Include rent & buy, Show everything.
- Save selected viewing mode to the user account.
- Discover strict mode must never silently bypass selected subscriptions.
- Adjacent, wildcard, and refill logic must not bypass provider filtering unless user explicitly opts in.
- Search always searches everything but labels availability.
- If selected genre plus selected services becomes thin, show a clear message like: You have reached the end of Netflix Horror.
- Offer explicit choices: Include other services, Include rent/buy, Broaden genres.
- Do not show unavailable/off-service content in primary strict Discover without user opt-in.

Acceptance tests:

- Netflix-only Horror shows only Netflix titles unless explicit opt-in.
- Prime-only Comedy shows only Prime titles unless explicit opt-in.
- Max-only TV uses Max/HBO TV provider data.
- No-subscriptions user sees appropriate setup or choice flow.

P1 - Tone Compatibility and Adjacent/Wildcard Quality

Why this matters: Exploration should feel related, not emotionally random.

Likely files / areas to inspect:

- backend/engine.py
- theme/tone enrichment logic

Implementation tasks:

- Adjacent/wildcard cards must pass both genre adjacency and tone compatibility.
- Horror/War/Thriller/Crime users should not receive light, uplifting, holiday, wholesome, feel-good, or family-safe content unless behavior supports it.
- Comedy/Romance/Family users should not receive brutal crime, war, horror, or dark content through exploration unless behavior supports it.
- For conflicts such as Horror + Comedy, relax carefully but avoid emotionally random content.
- Add debug fields: selected_genre, candidate_genre, candidate_tone, tone_compatible, rejection_reason.

Acceptance tests:

- Horror feed must not show Christmas/family/wholesome content.
- Romance feed must not show brutal horror/war as wildcard unless behavior clearly supports it.
- Documentary feed maintains subtype/tone continuity where possible.

P1 - UI Safety, Mobile, and Production Error Handling

Why this matters: Normal users should never see technical errors.

Likely files / areas to inspect:

- frontend API client
- AuthContext.jsx
- App error boundary
- Discover.jsx
- Search.jsx

Implementation tasks:

- Disable raw runtime overlays in production.
- Add React error boundaries and friendly recovery UI.
- Global API/network error handling should show user-friendly messages, not stack traces.
- Expired token/network failure should not crash the app.
- Add loading/empty states that explain what happened and what the user can do.
- Test mobile safe areas, bottom navigation, search close/back, filters, modals, focus states, loading skeletons, poor connection, and slow network.

Acceptance tests:

- Expired token during active swipe session recovers safely.
- Backend restart mid-session does not show raw crash overlay.
- Poor connection shows friendly retry state.

P1 - Analytics, Crash Reporting, Admin, and Trust Monitoring

Why this matters: After launch, you need visibility into what is working and what is failing.

Likely files / areas to inspect:

- analytics utility
- admin routes/dashboard
- frontend API client
- backend logging

Implementation tasks:

- Add analytics for signup, onboarding complete, search, search result click, swipe, save, skip, watched/seen, detail open, trailer open, empty state, refill, provider mismatch report, crash, session expiry.
- Track recommendation acceptance: saves per swipe, watches per swipe, detail opens per swipe, trailer opens per swipe.
- Track search health: no-result searches, search clicks, query types, most failed searches.
- Track provider health: mismatch reports, stale provider records, provider coverage by region.
- Admin dashboard should show users, catalogue stats, provider health, recommendation health, queue health, and user learning debug.
- Add frontend/backend crash reporting and failed API call logging.

Acceptance tests:

- Analytics event fires for each key action.
- Admin can see provider/catalogue/recommendation health.
- Crash/error test creates logged event.

P1 - Security, Abuse Protection, and Legal Requirements

Why this matters: Public beta needs basic safety and compliance.

Likely files / areas to inspect:

- auth routes
- admin routes
- legal pages/components
- settings/account page

Implementation tasks:

- Add signup/login rate limiting.
- Add brute-force protection on login/password reset.
- Add spam protection for contact form.
- Do not expose secrets, passwords, tokens, or admin credentials.
- Change default admin credentials via environment secrets before launch.
- Add Privacy Policy, Terms, Contact email, Delete Account, and data deletion/export flow.
- Add terms/privacy acceptance checkbox and timestamp stored to user account.

Acceptance tests:

- Rate limit repeated signup/login attempts.
- Non-admin cannot access admin routes.
- Account deletion removes or anonymizes user data according to policy.

P2 - Automated Tests, Manual Beta Protocol, and Final Pass Criteria

Why this matters: The app should not be called launch ready until the minimum tests pass.

Likely files / areas to inspect:

- tests/test_personalization_stress.py
- QA scripts
- admin dashboards

Implementation tasks:

- Run automated persona tests: modern English horror, superhero-only, true crime docs, casual mainstream, classics lover, international cinema, strict subscription-only, show-everything user.
- Run 100, 300, and 1000 swipe simulations across major genres and personas.
- Manual beta with 10-50 users first. Track bug, device, steps, expected, actual, screenshot, severity, category.
- Then expand to 50-100 testers before public launch.

- Do not keep adding features during beta stabilization. Fix blockers first.

Acceptance tests:

- Accounts: sign up, refresh, close/reopen, logout/login, 50 actions, all user data persists.
- Swipe speed: 100 rapid swipes without blank screen or waiting.
- Search: actor/title/genre/provider searches hit full catalogue with pagination and labels.
- Subscriptions: Netflix-only Discover shows only Netflix unless explicit opt-in is used.
- Providers: known Max/HBO, Netflix, Prime, Disney, Apple, rent/buy examples display correctly by region.
- Learning: after 50 actions, rejected language/decade/theme drops and liked niche rises.
- Card state: open MovieDetail and back without losing card, queue, toggles, or feed state.
- Catalogue: old/classic titles are searchable while Discover remains curated and modern-biased by default.
- Errors: expired token/network failure never shows raw Axios/runtime overlay.

Final Report Required From AI

FINAL REPORT REQUIRED FROM AI

When complete, return:

1. Files changed.
2. Database fields added or changed.
3. Environment secrets required.
4. Automated tests run.
5. Manual smoke tests run.
6. Passed tests.
7. Failed tests.
8. Remaining launch blockers.
9. Non-blocking improvements.
10. Honest beta readiness decision.

Do not claim launch-ready if accounts, search, provider accuracy, swipe speed, persistence, or personalization fail.